

Assignment_3.R

nishanthyn

2026-06-05

```
library(readxl)
library(forecast)
```

```
## Warning: package 'forecast' was built under R version 4.5.2
```

```
library(tseries)
```

```
## Warning: package 'tseries' was built under R version 4.5.2
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
# 1. Import the ABS workbook
```

```
retail_raw <- read_excel("/Users/nishanthyn/Desktop/Time_Series/Assignment_3/850101.xlsx", sheet = "Data")
```

```
## New names:
## * '' -> '...1'
## * '' -> '...2'
## * '' -> '...3'
## * '' -> '...4'
## * '' -> '...5'
## * '' -> '...6'
## * '' -> '...7'
## * '' -> '...8'
## * '' -> '...9'
## * '' -> '...10'
## * '' -> '...11'
## * '' -> '...12'
## * '' -> '...13'
## * '' -> '...14'
## * '' -> '...15'
## * '' -> '...16'
## * '' -> '...17'
## * '' -> '...18'
## * '' -> '...19'
## * '' -> '...20'
## * '' -> '...21'
## * '' -> '...22'
```

```
# 2. Extract date column and Total (Industry), Seasonally Adjusted series
date_index <- retail_raw[-c(1:10), 1][[1]]
turnover <- as.numeric(retail_raw[-c(1:10), 15][[1]])
turnover <- na.omit(turnover)
```

```
# 3. Create monthly time series object
retail_ts <- ts(turnover, start = c(1982, 4), frequency = 12)
```

```
# Figure 1: Descriptive plot
```

```
plot(retail_ts,
     main = "Australian Monthly Retail Turnover (Seasonally Adjusted)",
     ylab = "Turnover ($ million)",
     xlab = "Year")
```



```
# 4. Transformation
log_ts <- log(retail_ts)
diff_log_ts <- diff(log_ts)

# 5. Candidate models
fit_011 <- Arima(log_ts, order = c(0,1,1), include.drift = TRUE)
fit_111 <- Arima(log_ts, order = c(1,1,1), include.drift = TRUE)
fit_s011 <- Arima(log_ts, order = c(0,1,1),
                  seasonal = c(0,1,1), include.drift = TRUE)
```

```
## Warning in Arima(log_ts, order = c(0, 1, 1), seasonal = c(0, 1, 1),
## include.drift = TRUE): No drift term fitted as the order of difference is 2 or
## more.
```

```
fit_s111 <- Arima(log_ts, order = c(1,1,1),
                 seasonal = c(0,1,1), include.drift = TRUE)
```

```
## Warning in Arima(log_ts, order = c(1, 1, 1), seasonal = c(0, 1, 1),
## include.drift = TRUE): No drift term fitted as the order of difference is 2 or
## more.
```

```
# Compare models
```

```
AIC(fit_011, fit_111, fit_s011, fit_s111)
```

```
## Warning in AIC.default(fit_011, fit_111, fit_s011, fit_s111): models are not
## all fitted to the same number of observations
```

```
##           df      AIC
## fit_011    3 -2954.216
## fit_111    4 -2952.615
## fit_s011   3 -2837.895
## fit_s111   4 -2836.178
```

```
BIC(fit_011, fit_111, fit_s011, fit_s111)
```

```
## Warning in BIC.default(fit_011, fit_111, fit_s011, fit_s111): models are not
## all fitted to the same number of observations
```

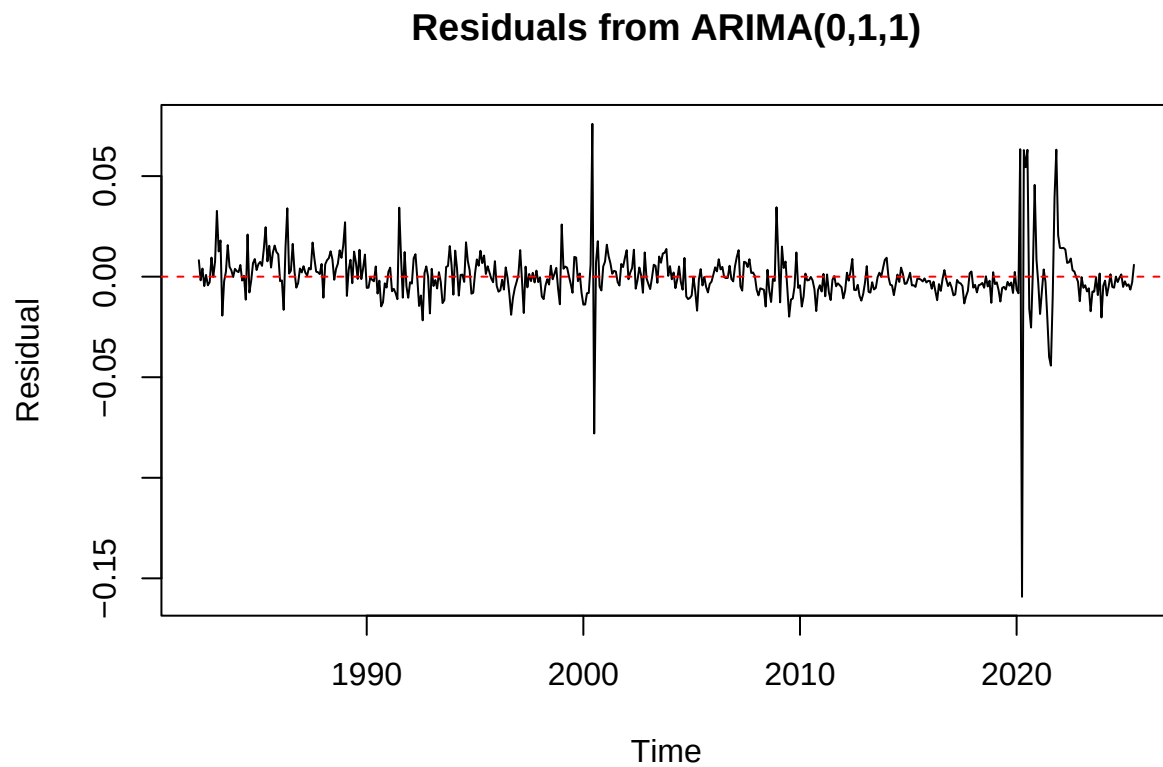
```
##           df      BIC
## fit_011    3 -2941.466
## fit_111    4 -2935.616
## fit_s011   3 -2825.216
## fit_s111   4 -2819.272
```

```
# Summary of selected model
```

```
summary(fit_011)
```

```
## Series: log_ts
## ARIMA(0,1,1) with drift
##
## Coefficients:
##           ma1    drift
##          -0.5095  0.0046
## s.e.      0.0350  0.0003
##
## sigma^2 = 0.0001938:  log likelihood = 1480.11
## AIC=-2954.22  AICc=-2954.17  BIC=-2941.47
##
## Training set error measures:
##           ME      RMSE      MAE      MPE      MAPE
## Training set 1.470736e-05 0.01388133 0.007895604 0.001628281 0.0831425
##           MASE      ACF1
## Training set 0.1423088 -0.01694005
```

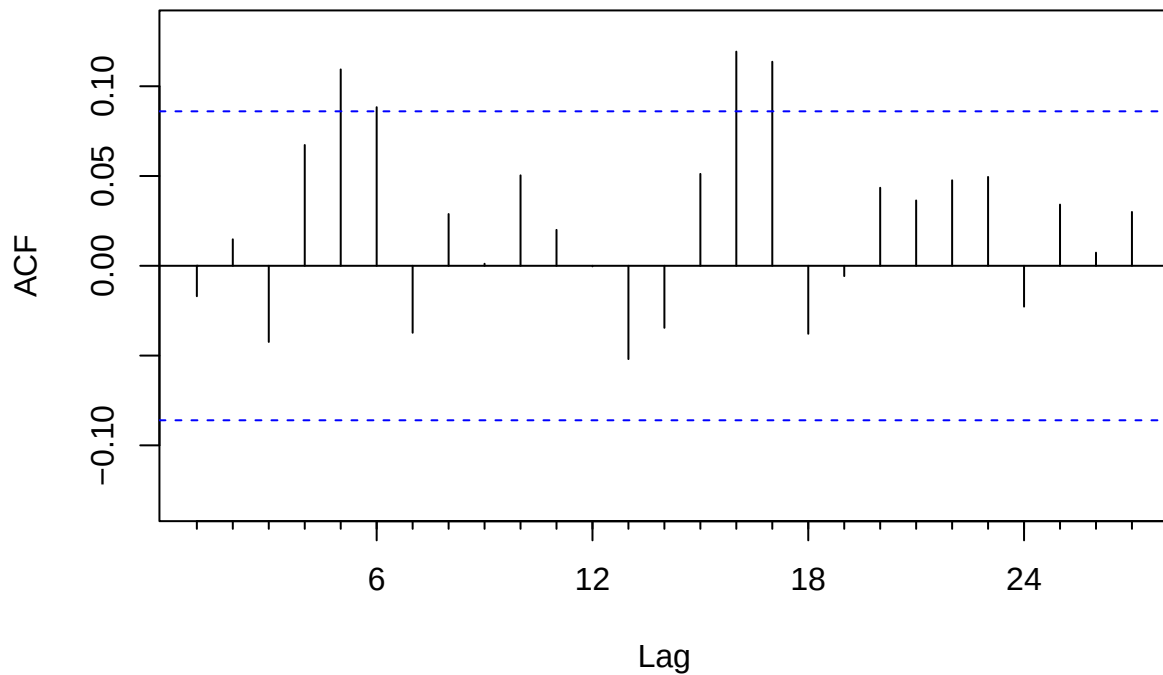
```
# 6. Residual diagnostics
# Residual time plot (optional to include in Section 6)
plot(residuals(fit_011), type = "l",
     main = "Residuals from ARIMA(0,1,1)",
     ylab = "Residual",
     xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```



```
# Figure 2: Residual ACF with 95% confidence bands
```

```
Acf(residuals(fit_011),
    main = "Residual ACF of ARIMA(0,1,1) with 95% Confidence Bands")
```

Residual ACF of ARIMA(0,1,1) with 95% Confidence Bands



```
# Ljung-Box test
Box.test(residuals(fit_011), lag = 12, type = "Ljung-Box")
```

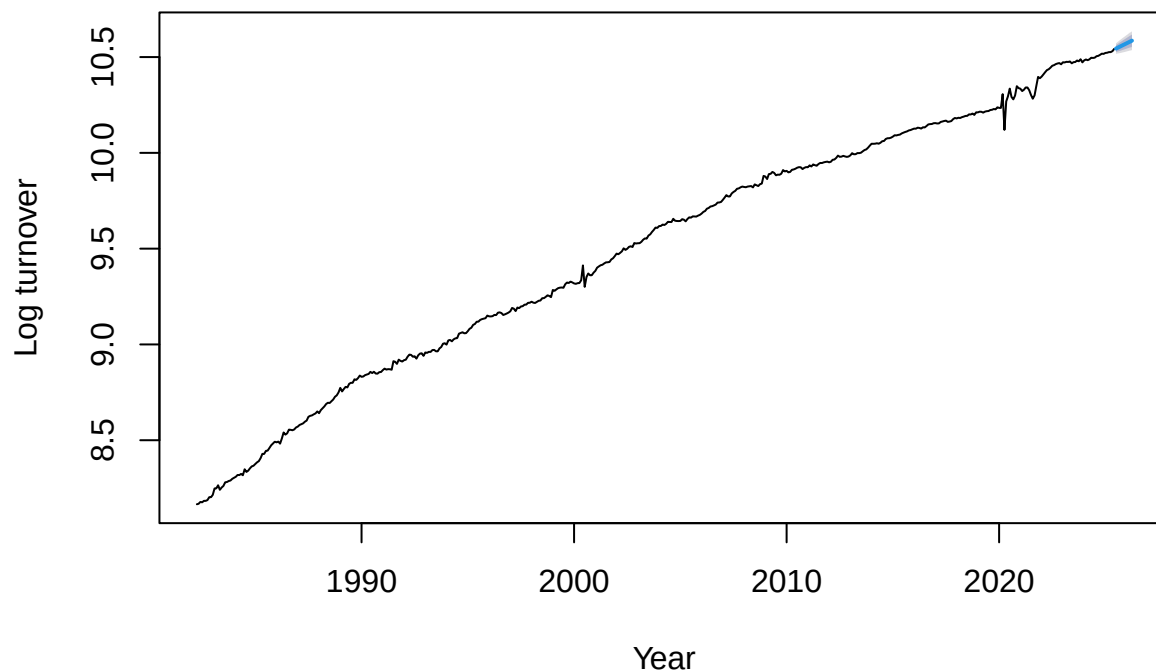
```
##
## Box-Ljung test
##
## data: residuals(fit_011)
## X-squared = 16.724, df = 12, p-value = 0.1603
```

```
# 7. Forecasting
fc <- forecast(fit_011, h = 10)
```

```
# Figure 3: Forecast plot
```

```
plot(fc,
      main = "10-Month Forecast of Australian Retail Turnover",
      xlab = "Year",
      ylab = "Log turnover")
```

10-Month Forecast of Australian Retail Turnover



```
# Forecast values on original scale
fc_original_million <- exp(fc$mean)
fc_original_bn <- fc_original_million / 1000

print(fc_original_million)
```

```
##           Jan      Feb      Mar      Apr May Jun      Jul      Aug      Sep
## 2025                               37966.68 38141.12 38316.36
## 2026 39025.40 39204.71 39384.83 39565.79
##           Oct      Nov      Dec
## 2025 38492.40 38669.26 38846.92
## 2026
```

```
print(fc_original_bn)
```

```
##           Jan      Feb      Mar      Apr May Jun      Jul      Aug      Sep
## 2025                               37.96668 38.14112 38.31636
## 2026 39.02540 39.20471 39.38483 39.56579
##           Oct      Nov      Dec
## 2025 38.49240 38.66926 38.84692
## 2026
```

```
# Optional: forecast intervals on original scale
forecast_table <- data.frame(
```

```

Month = time(fc$mean),
Point_Forecast_Million = as.numeric(exp(fc$mean)),
Lower_80_Million = as.numeric(exp(fc$lower[,1])),
Upper_80_Million = as.numeric(exp(fc$upper[,1])),
Lower_95_Million = as.numeric(exp(fc$lower[,2])),
Upper_95_Million = as.numeric(exp(fc$upper[,2]))
)

print(forecast_table)

```

```

##      Month Point_Forecast_Million Lower_80_Million Upper_80_Million
## 1  2025.500           37966.68         37295.31         38650.13
## 2  2025.583           38141.12         37390.65         38906.64
## 3  2025.667           38316.36         37493.33         39157.45
## 4  2025.750           38492.40         37601.72         39404.18
## 5  2025.833           38669.26         37714.75         39647.91
## 6  2025.917           38846.92         37831.70         39889.38
## 7  2026.000           39025.40         37952.02         40129.15
## 8  2026.083           39204.71         38075.31         40367.61
## 9  2026.167           39384.83         38201.24         40605.10
## 10 2026.250           39565.79         38329.57         40841.88
##      Lower_95_Million Upper_95_Million
## 1           36944.73         39016.89
## 2           36999.38         39318.09
## 3           37064.82         39610.15
## 4           37138.59         39895.56
## 5           37219.05         40175.97
## 6           37305.06         40452.51
## 7           37395.81         40726.01
## 8           37490.67         40997.11
## 9           37589.15         41266.30
## 10          37690.87         41533.98

```